

Reinforcement Learning

Dynamic Programming

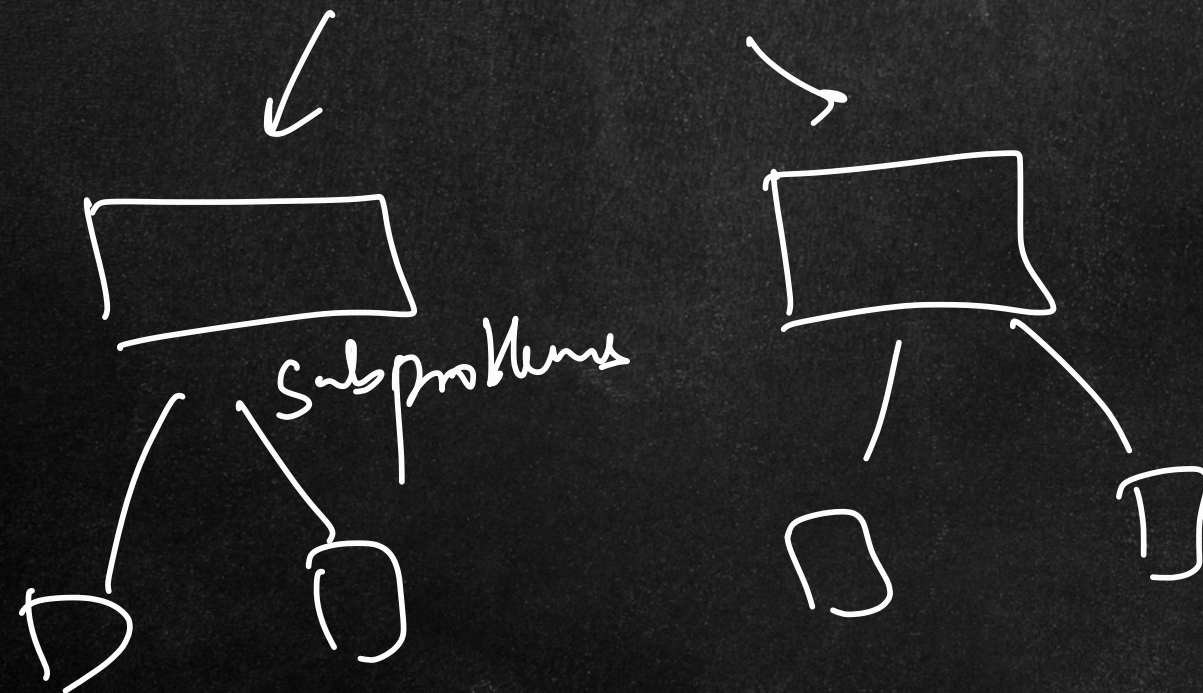
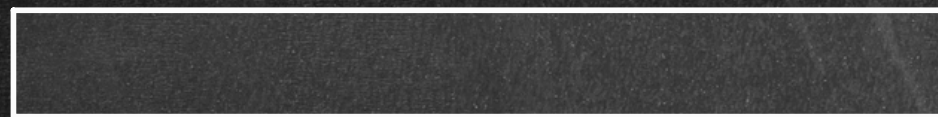
VF / policy

Optimal subproblems

① DP → DSA

② Monte Carlo methods

③ TD learning



	0	1	2	3
0				X
1			X	X
2	S		X	E

largest problem
↓
smallest problem

Reinforcement Learning

Dynamic Programming

↓
* model-free method
(complete knowledge of the env)

↓
 $V(s)$ $Q(s, a)$

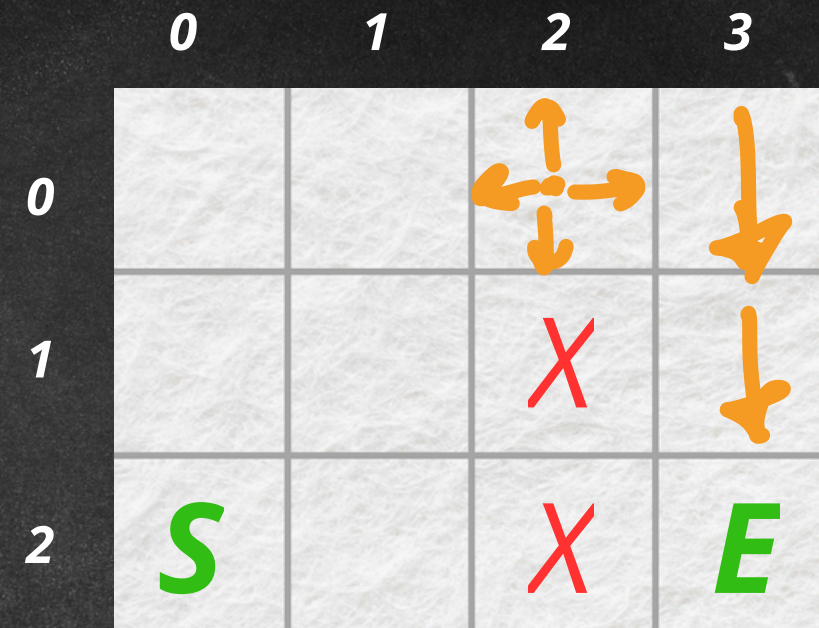
Bellman Equations

	0	1	2	3
0				
1			X	
2	S		X	E

Reinforcement Learning

Dynamic Programming

$$V(s) = \max_a (R(s, a) + \gamma \cdot V(s'))$$



Disadv.

- ① model-full
- ② scalability
- ③ computationally expensive for large MDPs

Reinforcement Learning

Monte Carlo Methods

↓
model-free algos

episodes → experienced samples

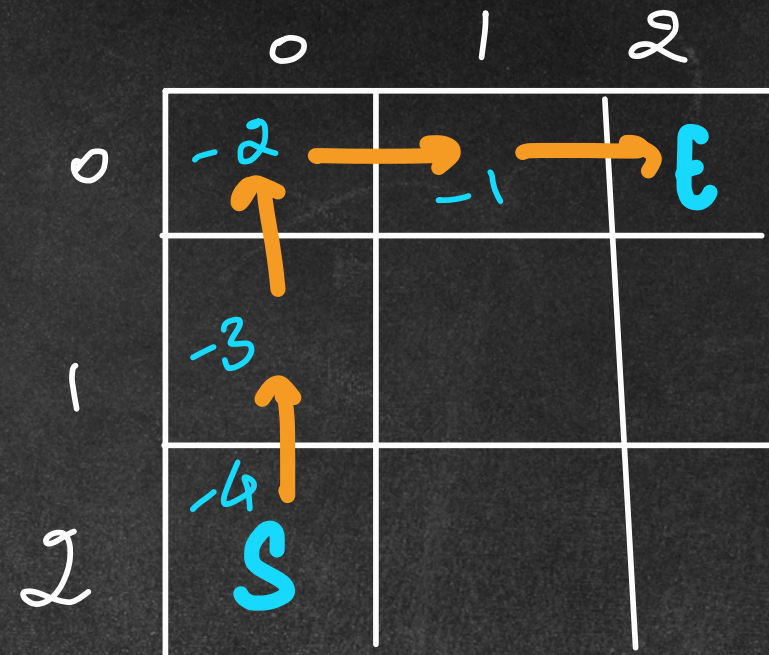
1st ep

step	state	action	reward
1	2,0	↑	-1
2	1,0	↑	-1
3	0,0	→	-1
4	0,1	→	0

$\gamma = 1$

$$G_t = R_{t+1} + \cancel{\gamma} \cdot R_{t+2} + \cancel{\gamma^2} \cdot R_{t+3}$$

$$G_t = -1 + (-1) + -1 = -3$$



update step

$$V(2,0) = -4$$

$$V(1,0) = -3$$

$$V(0,0) = -2$$

⋮

Reinforcement Learning

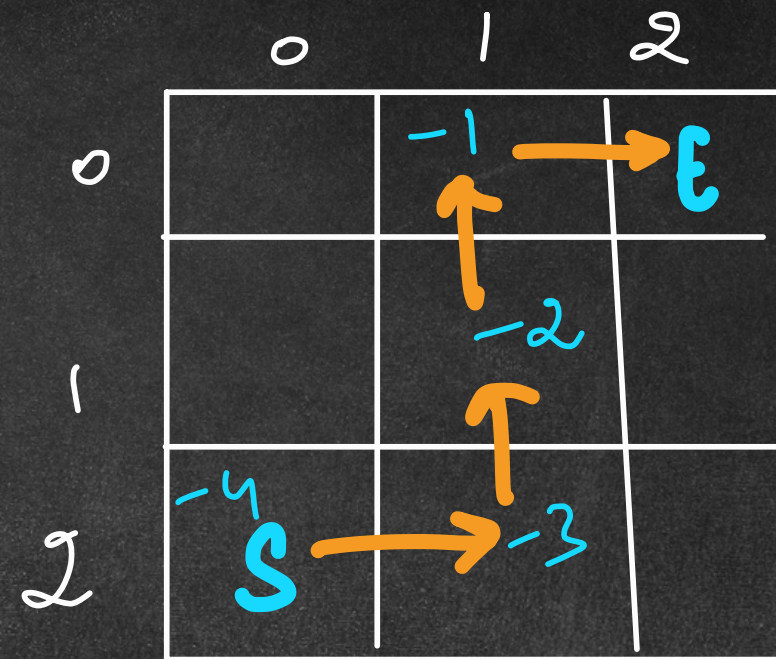
Monte Carlo Methods

ex 2

step	state	action	reward
1	(2,0)	→	-1
2	2,1	↑	-1
3	1,1	↑	-1
4	0,1	→	-1

$$V(2,0) = \frac{-4 + (-4)}{2} = -4$$

$$V(2,1) = \frac{0 + (-3)}{2} = -1.5$$



0			E
1	-2.7	-2.5	
2	S	-3.5	

Reinforcement Learning

Monte Carlo Methods

- ① episode based $\rightarrow$ long episodes
 $\downarrow$
slow learning
 - ② continuous problems $\rightarrow$ MCMX
 - ③ learning $\rightarrow$ avg. consequences of individual path
 $\downarrow$
each step
- episode
long. consequence

Reinforcement Learning

Temporal Difference (TD) Learning

↓ model-free (steps)

↑ update rules

learning rate

step 1

$$V(s) = V(s) + \alpha \cdot [R + \gamma \cdot V(s') - V(s)]$$

reward

state val of next state

state val of cur state

TD
TD error

$$\alpha = 0.5, \gamma = 1$$

$$V(2,0) = V(2,0) + 0.5 \cdot [-1 + 1 \cdot V(1,0) - V(2,0)]$$

$$V(s) = 0.5 \cdot (-1) = -0.5$$

	0	1	2
0	↑ S		E
1			
2			

	0	1	2
0	0	0	0
1	0	0	0
2	-0.5	0	0

Reinforcement Learning

Temporal Difference (TD) Learning

$$V(s) = V(s) + \alpha [R + \gamma V(s') - V(s)]$$

$$V(1,0) = V(0,0) + 0.5 [-1 + 1 \cdot V(0,0) - V(1,0)]$$

$$V(1,0) = -0.5$$

① SARSA

② Q-learning

① step wise learning

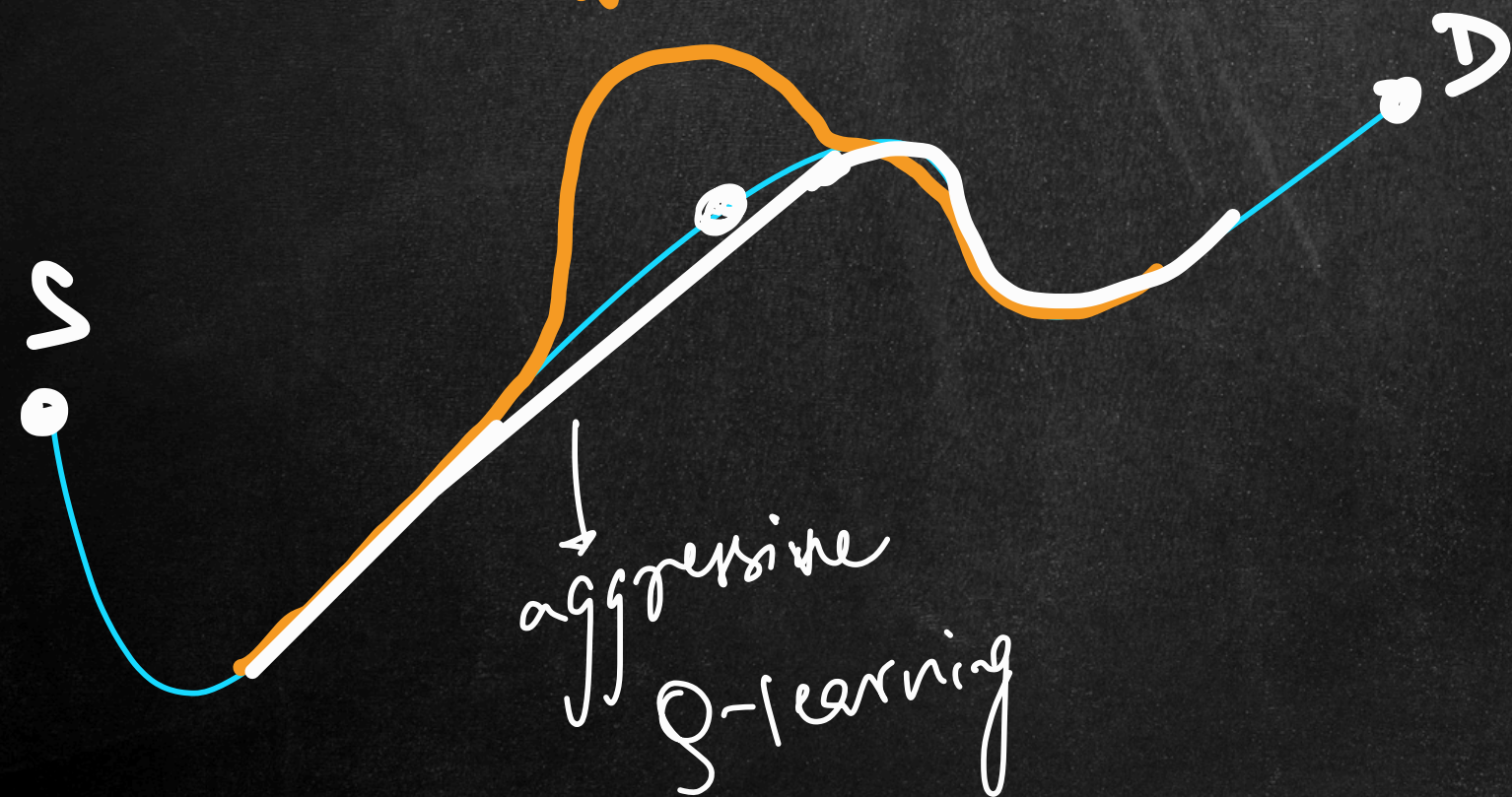
faster learning $\Rightarrow$ faster convergence

	0	1	2
0			E
1			
2	S		

$V(s)$		
0	0	0
-0.5	0	0
-0.5	0	0

Reinforcement Learning

SARSA



SARSA

↓ safer

↓ aggressive
Q-learning

learn to act in an
env
goal: $\uparrow$ total rewards

- ① SARSA
- ② Q-learning
- ① TD methods
- ② model-free
- ③ Q-value or Action value

Reinforcement Learning $\rightarrow$ on-policy algo

SARSA $\rightarrow$ State, Action, Reward, Next State, Next Action

- ① current state (S_t)
- ② " action (A_t) $\Rightarrow$ Policy (ϵ -greedy)

- ③ Take action
Reward $\rightarrow R_{t+1}$
Next state $\rightarrow S_{t+1}$

- ④ Next action (A_{t+1}) $\rightarrow$ Policy(S_{t+1}, A_{t+1})

- ⑤ updating Q-value $Q(s, a)$
SARSA update

$$Q(s, a) = Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$$

learning rate γ

$Q(\text{state, action})$

1000 episodes

10% explore
90% exploit

Reinforcement Learning

Q-Learning $\rightarrow$ off-policy

- ① curr state s_t
- ② " action $A_t \rightarrow$ policy (ϵ -greedy)
- ③ Take action
Reward r_{t+1}
next state s_{t+1}
- ④ next action (A_{t+1}) $\Rightarrow$ on the basis of Q-val
next best action pick for next state
- ⑤ update Q-val # Q-learning update

$$Q(s, a) = Q(s, a) + \alpha \cdot \left[R + \gamma \cdot \max_a Q(s', a) - Q(s, a) \right]$$

shoot a path

optimal Q-val

Reinforcement Learning

Cliff Walking Problem

0	1	2	3	4	5	6	7	8	9	10	11
12	13	14	15								
S	C	C	C	C	*	C	C	C	C	C	E

$(3, 0)$

cliff

State: $4 \times 12 = 48$ cells (row, col)

(single num) $\rightarrow$ state = $\text{row} \times 12 + \text{col}$

$$\text{state of } S = 3 \times 12 + 0 = \underline{\underline{36}}$$

env: 4×12 grid

Actions: $\uparrow, \downarrow, \leftarrow, \rightarrow \in \{0, 3\}$

Reward: normal cells $\rightarrow -1$
target " $\rightarrow 0$
cliff $\rightarrow -100$

Reinforcement Learning

Gymnasium

env object

-
- ```
graph TD; Env["env object"] --> Create["(1) create -> make()"]; Env --> Show["(2) show -> render()"]; Env --> Reset["(3) reset -> reset()"]; Env --> Close["(4) close -> close()"]; Env --> Action["(5) action -> step()"];
```
- (1) create → make()
  - (2) show → render()
  - (3) reset → reset()
  - (4) close → close()
  - (5) action → step()



# Reinforcement Learning

## SARSA Implementation

$$Q(s, a) = Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$$

episodes = 500

alpha

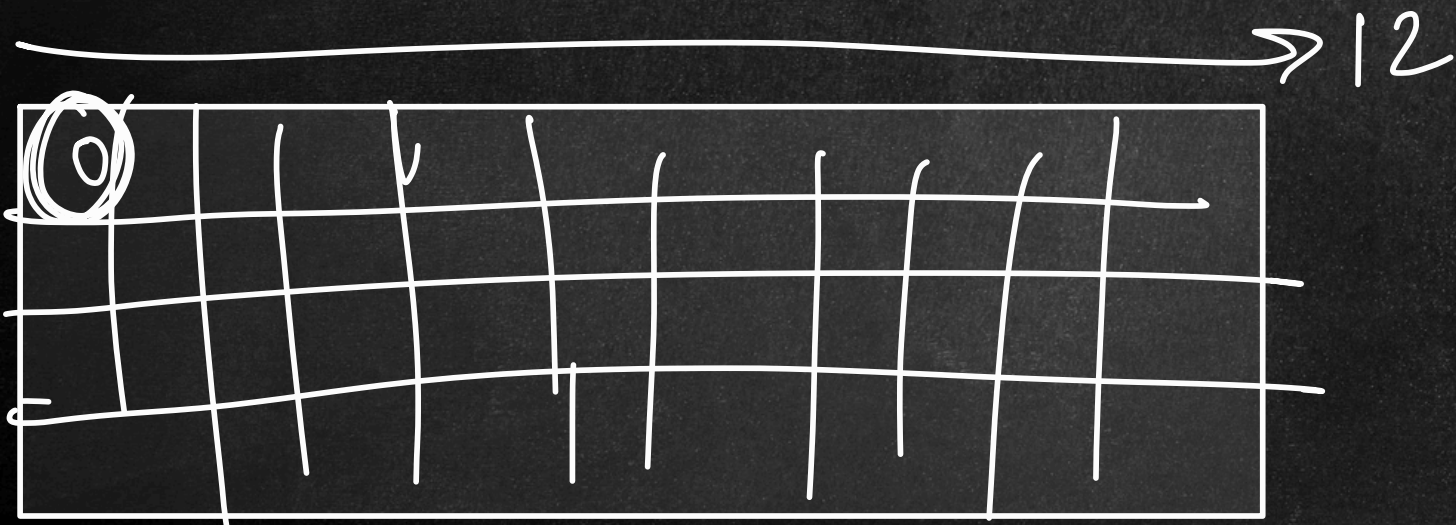
gamma

Policy  $\rightarrow$   $\epsilon$  greedy

States = 48      action = 4

Q-value  
 $\rightarrow$  48 x 4

- ①  $\alpha = 0.5$
- ②  $\gamma = 0.99$
- ③ episodes = 500
- ④  $\epsilon = 0.1$





# Reinforcement Learning

## SARSA Implementation

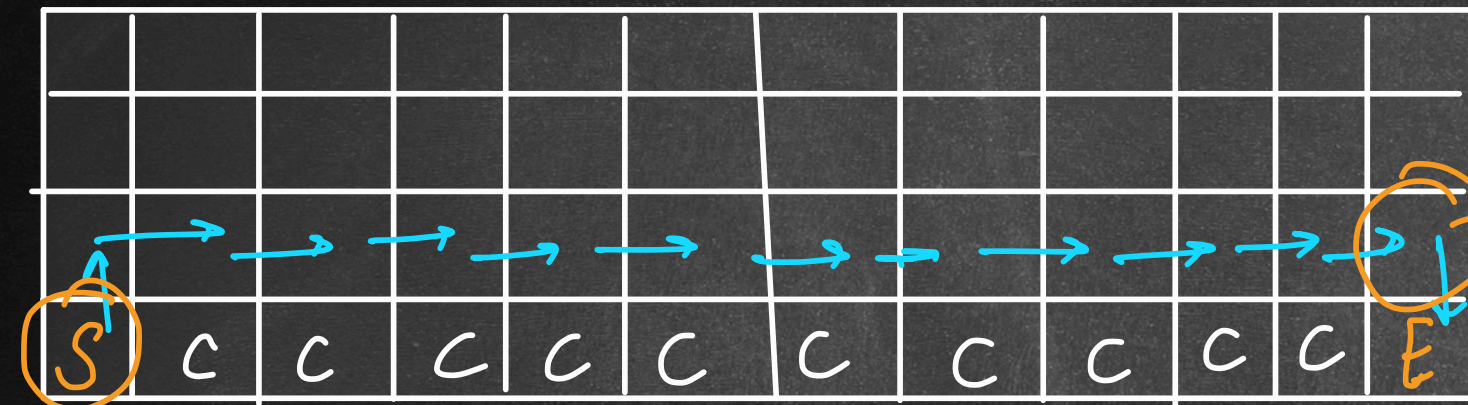
gym  $\rightarrow$  humans  
training  $\rightarrow$  slow

done  $\left\{ \begin{array}{l} \text{terminate} \rightarrow \text{agent reached } E \\ \text{truncate} \rightarrow \text{cut short} \end{array} \right.$   
max time  
" path length



# Reinforcement Learning

## Q-learning Implementation



$$\underline{Q(s,a)} \leftarrow \alpha * [R + \gamma \cdot \max_a Q(s',a) - Q(s,a)]$$

↓  
original  
Q-value

state = 35

2,11 action = down ↓ (2)

$$\begin{aligned} & 2 * (2 + 1) \\ &= 2 * 3 \\ &= 6 \end{aligned}$$

state = 36

action = up ↑  
(0)

Q-table